

FILE OPERATIONS

1. Write a C program to create a new file using the open() system call. What flags and permissions are required?

PROGRAM:

```
/*
Program to open a file
Author   : MUTHUGANESH S
Date     : 01/02/2026
Filename : Open.c
*/

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {

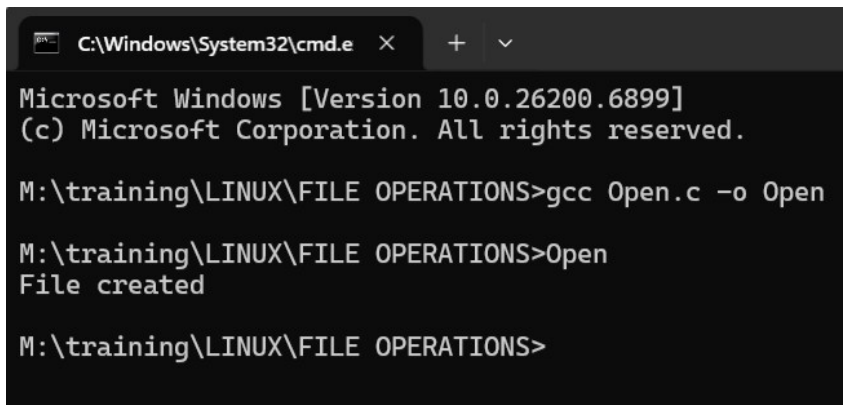
    int fd = open("file.txt", O_CREAT | O_WRONLY, 0644);

    //O_CREAT → create file if not present
    //O_WRONLY → write only

    if(fd == -1) {
        perror("open");
        return 1;
    }

    printf("File created\n");
    close(fd);
}
```

OUTPUT:



```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.6899]
(c) Microsoft Corporation. All rights reserved.

M:\training\LINUX\FILE OPERATIONS>gcc Open.c -o Open

M:\training\LINUX\FILE OPERATIONS>Open
File created

M:\training\LINUX\FILE OPERATIONS>
```

2. Implement a program that writes "Hello, Linux!" to a file using write(). How do you handle errors?

PROGRAM:

```
/*
Program to illustarte write()
Author   : MUTHUGANESH S
Date     : 01/02/2026
Filename : Write.c
*/

//Header files
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {

    int fd = open("hello.txt", O_CREAT | O_WRONLY, 0644);

    if(fd < 0) {
        perror("open");
        return 1;
    }

    char msg[] = "Hello, Linux!\n";

    if(write(fd, msg, sizeof(msg)) < 0) {
        perror("write");
    }

    close(fd);
}
```

OUTPUT:

```
M:\training\LINUX\FILE OPERATIONS>gcc Write.c -o Write
M:\training\LINUX\FILE OPERATIONS>Write
```

3. Write a C program to read the contents of a file using read() and display it on the console

PROGRAM:

```
/*
Program to read a file
Author   : MUTHUGANESH S
Date     : 01/02/2026
Filename : Read.c
*/

//Header files
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {

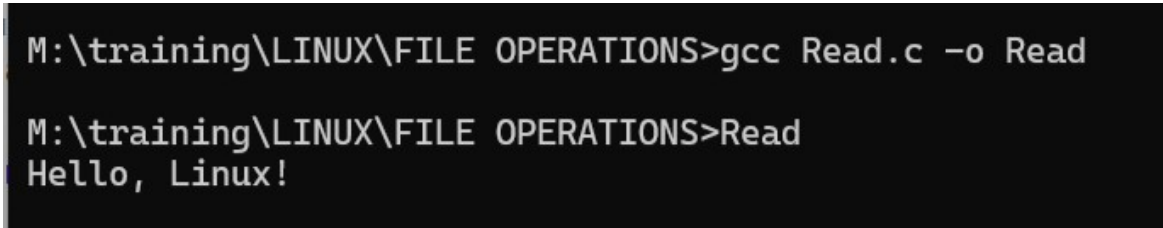
    char buffer[100];
    int fd = open("hello.txt", O_RDONLY);

    int n = read(fd, buffer, sizeof(buffer));

    write(1, buffer, n);

    close(fd);
}
```

OUTPUT:



```
M:\training\LINUX\FILE OPERATIONS>gcc Read.c -o Read
M:\training\LINUX\FILE OPERATIONS>Read
Hello, Linux!
```

4. Demonstrate how to use lseek() to move the file pointer to the beginning, middle, and end of a file.

PROGRAM:

```
/*
Program to illustarte lseek()
Author   : MUTHUGANESH S
Date     : 01/02/2026
```

```

Filename : lseek.c
*/

//Header files
#include <fcntl.h>
#include <unistd.h>

int main(){

int fd=open("hello.txt",O_RDONLY);

lseek(fd,0,SEEK_SET);//start of file
lseek(fd,5,SEEK_SET);//5th byte
lseek(fd,0,SEEK_END);//end of file

close(fd);
}

```

OUTPUT:

```

M:\training\LINUX\FILE OPERATIONS>gcc Lseek.c -o Lseek
M:\training\LINUX\FILE OPERATIONS>Lseek
M:\training\LINUX\FILE OPERATIONS>

```

5. Write a program to copy the contents of one file to another using only system calls (open, read, write, close).

PROGRAM:

```

/*
Program to copy using System calls
Author   : MUTHUGANESH S
Date     : 02/02/2026
Filename : Copy.c
*/

//Header files
#include <fcntl.h>
#include <unistd.h>

int main(){

char buf[1024];

```

```

int src=open("a.txt",O_RDONLY);
int dest=open("b.txt",O_CREAT|O_WRONLY,0644);

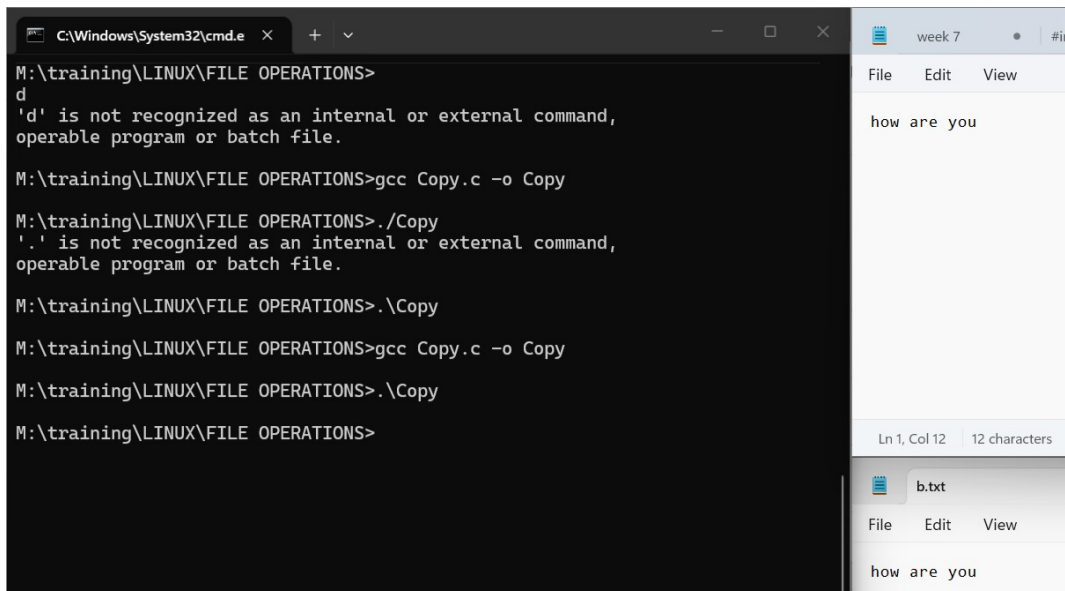
int n;

while((n=read(src,buf,sizeof(buf)))>0)
write(dest,buf,n);

close(src);
close(dest);
}

```

OUTPUT:



The screenshot shows a Windows command prompt window with the following commands and output:

```

M:\training\LINUX\FILE OPERATIONS>d
'd' is not recognized as an internal or external command,
operable program or batch file.

M:\training\LINUX\FILE OPERATIONS>gcc Copy.c -o Copy

M:\training\LINUX\FILE OPERATIONS>./Copy
'.' is not recognized as an internal or external command,
operable program or batch file.

M:\training\LINUX\FILE OPERATIONS>.\Copy

M:\training\LINUX\FILE OPERATIONS>gcc Copy.c -o Copy

M:\training\LINUX\FILE OPERATIONS>.\Copy

M:\training\LINUX\FILE OPERATIONS>

```

To the right of the command prompt, a text editor window is open, showing the contents of a file named 'b.txt'. The file contains the text 'how are you'.

6. Show how to change the permissions of a file using the chmod() system call.

PROGRAM:

```

/*
Program change permission
Author   : MUTHUGANESH S
Date     : 02/02/2026
Filename : Perm.c
*/

//Header files
#include <sys/stat.h>

int main(){

```

```
//change permission
chmod("file.txt",0777);

}
```

OUTPUT:

```
M:\training\LINUX\FILE OPERATIONS>gcc Perm.c -o Perm
M:\training\LINUX\FILE OPERATIONS>Perm
M:\training\LINUX\FILE OPERATIONS>
```

7. Write a C program to delete a file using the unlink() system call.

PROGRAM:

```
/*
Program to delete a file using System calls
Author   : MUTHUGANESH S
Date     : 02/02/2026
Filename : Del.c
*/

//Header files
#include <unistd.h>

int main(){

    //delete file
    unlink("file.txt");

}
```

OUTPUT:

```
M:\training\LINUX\FILE OPERATIONS>gcc Del.c -o Del
M:\training\LINUX\FILE OPERATIONS>.\Del
M:\training\LINUX\FILE OPERATIONS>|
```

8. Implement a program that creates a directory using `mkdir()` and removes it using `rmdir()`.

PROGRAM:

```
/*
Program to Create and remove directory using System calls
Author   : MUTHUGANESH S
Date     : 02/02/2026
Filename : Dir.c
*/

#include <stdio.h>
#include <direct.h>

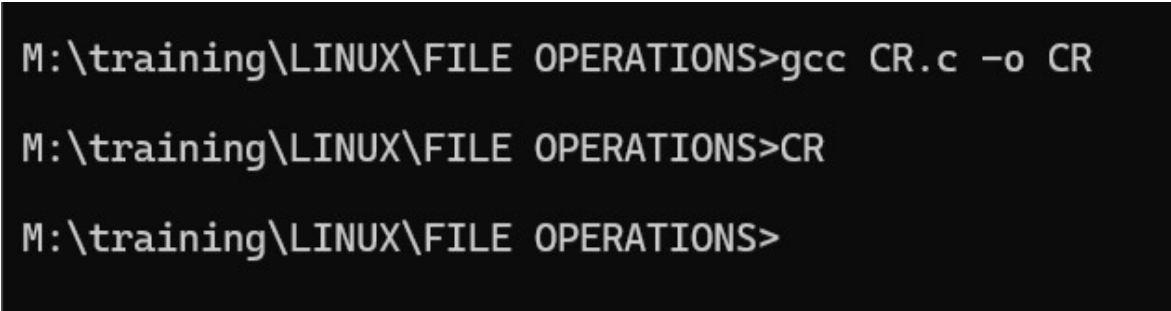
int main(){

    mkdir("testdir");

    rmdir("testdir");

    return 0;
}
```

OUTPUT:



```
M:\training\LINUX\FILE OPERATIONS>gcc CR.c -o CR
M:\training\LINUX\FILE OPERATIONS>CR
M:\training\LINUX\FILE OPERATIONS>
```

9. Write a program to list all files in a directory using `opendir()`, `readdir()`, and `closedir()`.

PROGRAM:

```
/*
Program to list directory using System calls
Author   : MUTHUGANESH S
Date     : 02/02/2026
Filename : Dir.c
*/

//Header files
```

```

#include <dirent.h>
#include <stdio.h>

int main(){

//open directory
DIR *d=opendir(".");

//d is a pointer to directory
struct dirent *dir;

while((dir=readdir(d))!=NULL)
printf("%s\n",dir->d_name);

//close directory
closedir(d);

}

```

OUTPUT:

```

M:\training\LINUX\FILE OPERATIONS>gcc CR.c -o Dir.c -o Dir
M:\training\LINUX\FILE OPERATIONS>Dir
Volume in drive M is New Volume
Volume Serial Number is B4CE-8090

Directory of M:\training\LINUX\FILE OPERATIONS

16-03-2026  16:27    <DIR>          .
16-03-2026  14:47    <DIR>          ..
16-03-2026  15:21    <DIR>        .vscode
16-03-2026  15:08             13 a.txt
16-03-2026  15:09             13 b.txt
16-03-2026  15:13           389 Copy.c
16-03-2026  15:09       41,530 Copy.exe
16-03-2026  15:22           260 CR.c
16-03-2026  15:23       40,850 CR.exe
16-03-2026  15:19           223 Del.c
16-03-2026  15:17       40,766 Del.exe
16-03-2026  16:27           391 Dir.c
16-03-2026  16:27       40,850 Dir.exe
16-03-2026  14:59            16 hello.txt
16-03-2026  15:04           344 Lseek.c
16-03-2026  15:03       41,446 Lseek.exe
16-03-2026  14:55           445 Open.c
16-03-2026  14:54       41,531 Open.exe
16-03-2026  15:14           216 Perm.c
16-03-2026  15:14       40,765 Perm.exe
16-03-2026  14:58           359 Read.c
16-03-2026  14:59       41,530 Read.exe
16-03-2026  14:58           471 Write.c
16-03-2026  14:59       41,532 Write.exe
                21 File(s)          373,940 bytes
                3 Dir(s)  135,442,898,944 bytes free

M:\training\LINUX\FILE OPERATIONS>

```


10. Demonstrate how to use stat() to display file metadata (size, permissions, modification time).

PROGRAM:

```
/*
Program to get size and permissions of a file
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : stat.c
*/

//Header files
#include <sys/stat.h>
#include <stdio.h>

int main(){

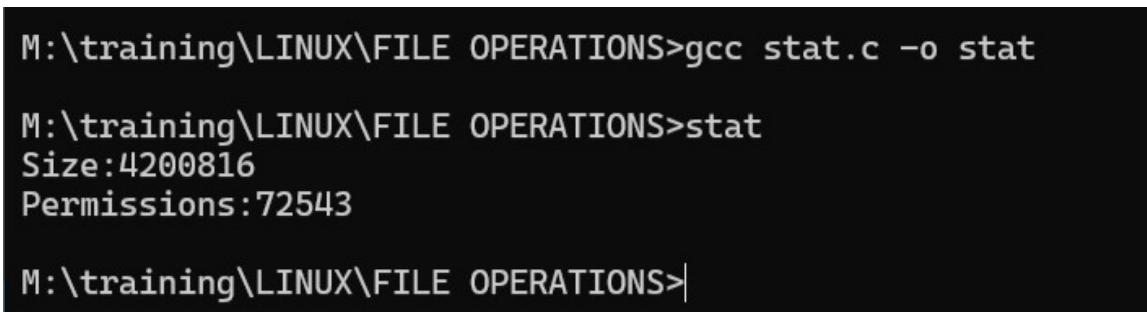
    struct stat s;

    //get size and permissions
    stat("file.txt",&s);

    printf("Size:%ld\n",s.st_size);
    printf("Permissions:%o\n",s.st_mode);

}
```

OUTPUT:



```
M:\training\LINUX\FILE OPERATIONS>gcc stat.c -o stat

M:\training\LINUX\FILE OPERATIONS>stat
Size:4200816
Permissions:72543

M:\training\LINUX\FILE OPERATIONS>|
```

11. Write a C program that demonstrates a race condition when two processes write to the same file simultaneously. How can you prevent this using file locks?

PROGRAM:

```
/*
Program to demonstrate race condition
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : race.c
*/

//Header files
```

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    int fd = open("shared.txt", O_WRONLY | O_CREAT | O_APPEND,
0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    pid_t pid = fork();

    for(int i = 0; i < 10; i++)
    {
        if(pid == 0)
        {
            write(fd, "Child writing\n", 14);
        }
        else
        {
            write(fd, "Parent writing\n", 15);
        }

        usleep(100000);
    }

    close(fd);
}

```

```

/*
Program to implement file locking
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : lock.c
*/

```

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

```

```

void lock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLKW, &lock);
}

void unlock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLK, &lock);
}

int main()
{
    int fd = open("shared.txt", O_WRONLY | O_CREAT | O_APPEND,
0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

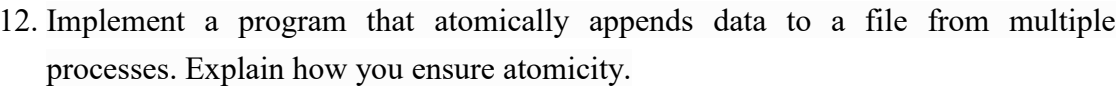
    pid_t pid = fork();

    for(int i = 0; i < 10; i++)
    {
        lock_file(fd);

        if(pid == 0)
        {
            write(fd, "Child writing\n", 14);
        }
        else
        {
            write(fd, "Parent writing\n", 15);
        }
    }
}

```

OUTPUT:



PROGRAM:

```

/*
Program to implement atomic write
Author      : MUTHUGANESH S
Date       : 03/02/2026
Filename    : atomic.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

int main()

```

```

{
    int fd = open("atomic.txt", O_WRONLY | O_CREAT | O_APPEND,
0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    pid_t pid = fork();

    for(int i = 0; i < 10; i++)
    {
        char buffer[50];

        if(pid == 0)
        {
            sprintf(buffer, "Child process writing %d\n", i);
        }
        else
        {
            sprintf(buffer, "Parent process writing %d\n", i);
        }

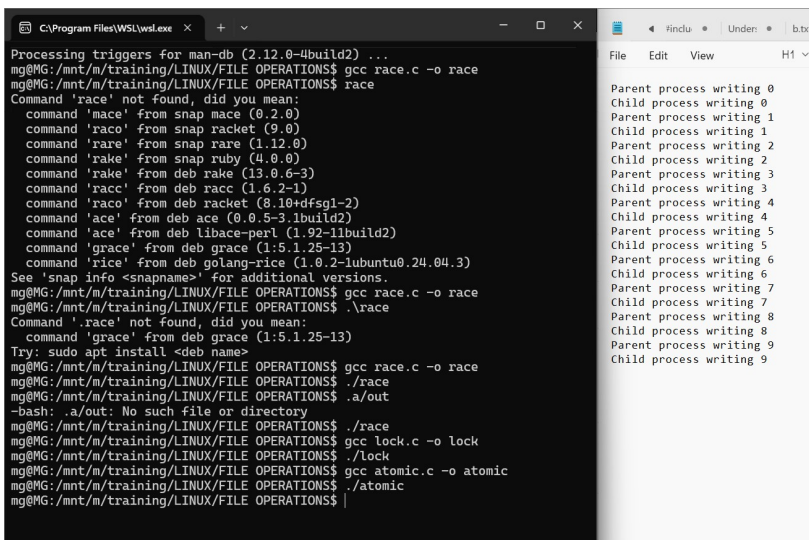
        write(fd, buffer, strlen(buffer));

        sleep(1);
    }

    close(fd);
    return 0;
}

```

OUTPUT:



```

C:\Program Files\WSL\wsl.exe x + v
Processing triggers for man-db (2.12.0-4build2) ...
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc race.c -o race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./race
Command 'race' not found, did you mean:
  command 'racc' from snap racc (0.2.0)
  command 'raco' from snap racket (9.0)
  command 'rare' from snap rare (1.12.0)
  command 'rake' from snap ruby (4.0.0)
  command 'rake' from deb rake (13.0.6-3)
  command 'racc' from deb racc (1.6.2-1)
  command 'raco' from deb racket (8.10+dfsg1-2)
  command 'ace' from deb ace (0.0.5-3.1build2)
  command 'ace' from deb libace-perl (1.92-11build2)
  command 'grace' from deb grace (1:5.1.25-13)
  command 'rice' from deb golang-rice (1.0.2-lubuntu0.24.04.3)
See 'snap info <snapnames>' for additional versions.
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc race.c -o race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./race
Command 'race' not found, did you mean:
  command 'grace' from deb grace (1:5.1.25-13)
Try: sudo apt install <deb name>
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc race.c -o race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./a.out
-bash: ./a.out: No such file or directory
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc lock.c -o lock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./lock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc atomic.c -o atomic
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./atomic
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$

```

13. Use lseek() to create a sparse file. Show how to verify the file's actual disk usage versus its reported size.

PROGRAM:

```
/*
Program to create a sparse file
Author   : MUTHUGANESH S
Date     : 04/02/2026
Filename : sparse.c
*/

//Header files
#include <fcntl.h>
#include <unistd.h>

int main(){

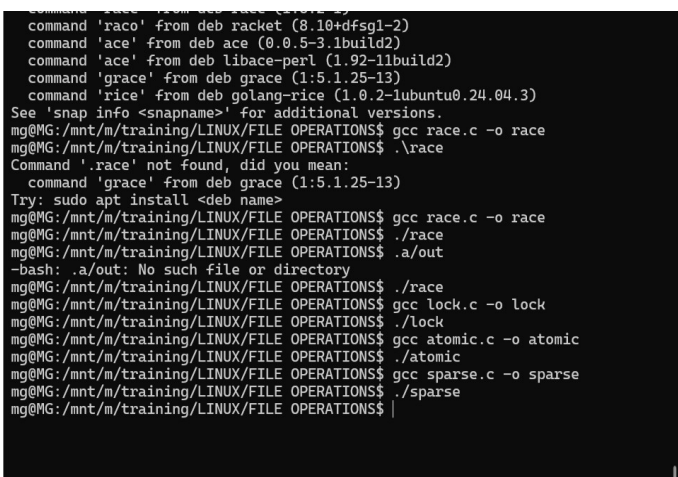
//Creating a sparse file
int fd=open("sparse.txt",O_CREAT|O_WRONLY,0644);

lseek(fd,10000,SEEK_SET);

write(fd,"A",1);

close(fd);
}
```

OUTPUT:



A terminal window showing a series of commands and their outputs. The user is in a directory /mnt/m/training/LINUX/FILE OPERATIONS. They first try to run 'race' but get an error. Then they run 'gcc race.c -o race' and './race', which also fails. They then run 'sudo apt install <deb name>' and finally 'gcc lock.c -o lock', 'gcc atomic.c -o atomic', and 'gcc sparse.c -o sparse'. The terminal output shows the command history and the results of these commands.

```
command 'race' from deb racket (8.10+dfsg1-2)
command 'ace' from deb ace (0.0.5-3.1build2)
command 'ace' from deb libace-perl (1.92-11build2)
command 'grace' from deb grace (1:5.1.25-13)
command 'rice' from deb golang-rice (1.0.2-1ubuntu0.24.04.3)
See 'snap info <snapname>' for additional versions.
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc race.c -o race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./race
Command 'race' not found, did you mean:
command 'grace' from deb grace (1:5.1.25-13)
Try: sudo apt install <deb name>
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc race.c -o race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./out
-bash: ./out: No such file or directory
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./race
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc lock.c -o lock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./lock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc atomic.c -o atomic
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./atomic
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc sparse.c -o sparse
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./sparse
```

14. Write a program that uses `fcntl()` to set a write lock on a region of a file. How do you handle lock contention between processes?

PROGRAM:

```
/*
Program to implement file locking
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : flock.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

int main()
{
    int fd;
    struct flock lock;

    fd = open("data.txt", O_RDWR | O_CREAT, 0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 50;
    lock.l_pid = getpid();

    printf("Process %d trying to acquire lock...\n", getpid());

    if(fcntl(fd, F_SETLKW, &lock) == -1)
    {
        perror("fcntl");
        exit(1);
    }

    printf("Lock acquired by process %d\n", getpid());
    printf("Writing to file...\n");

    write(fd, "Locked region write\n", 20);
}
```

```

    sleep(10);

    lock.l_type = F_UNLCK;
    fcntl(fd, F_SETLK, &lock);

    printf("Lock released by process %d\n", getpid());

    close(fd);

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./sparse
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc flock.c -o flock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./flock
Process 1804 trying to acquire lock...
Lock acquired by process 1804
Writing to file...
Lock released by process 1804
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$

```

15. Demonstrate how to use `fcntl()` to implement a read lock and a write lock, and explain the difference in behavior when multiple processes access the file.

PROGRAM:

```

/*
Program to read and write lock
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : rwlock.c
*/

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

int main()
{
    int fd;
    struct flock lock;

    fd = open("data.txt", O_RDWR | O_CREAT, 0644);

    if(fd < 0)

```



```

{
    perror("open");
    exit(1);
}

lock.l_type = F_WRLCK;
lock.l_whence = SEEK_SET;
lock.l_start = 0;
lock.l_len = 50;
lock.l_pid = getpid();

printf("Process %d trying to acquire lock...\n", getpid());

if(fcntl(fd, F_SETLKW, &lock) == -1)
{
    perror("fcntl");
    exit(1);
}

printf("Lock acquired by process %d\n", getpid());
printf("Writing to file...\n");

write(fd, "Locked region write\n", 20);

sleep(10);

lock.l_type = F_UNLCK;
fcntl(fd, F_SETLK, &lock);

printf("Lock released by process %d\n", getpid());

close(fd);

return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc rwlock.c -o rwlock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./rwlock
Process 1852 trying to acquire lock...
Lock acquired by process 1852
Writing to file...
Lock released by process 1852
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$

```

16. Create a program that uses lseek() to read and write at random offsets in a file.
How do you ensure data consistency?

PROGRAM:

```
/*
Program to read and write using lseek
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : rwlseek.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    int fd;
    char buffer[50];

    fd = open("random.txt", O_RDWR | O_CREAT, 0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    strcpy(buffer, "HELLO");

    lseek(fd, 10, SEEK_SET);
    write(fd, buffer, strlen(buffer));

    lseek(fd, 30, SEEK_SET);
    write(fd, "LINUX", 5);

    lseek(fd, 10, SEEK_SET);
    read(fd, buffer, 5);
    buffer[5] = '\0';

    printf("Data at offset 10: %s\n", buffer);

    lseek(fd, 30, SEEK_SET);
    read(fd, buffer, 5);
    buffer[5] = '\0';
```

```

    printf("Data at offset 30: %s\n", buffer);

    close(fd);

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc rwlseek.c -o rwlseek
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./rwlseek
Data at offset 10: HELLO
Data at offset 30: LINUX
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ |

```

17. Write a C program that uses advisory file locking (fcntl) to synchronize access between a parent and child process. What happens if one process ignores the lock?

PROGRAM:

```

/*
Program to implement parent child synchronization
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : parent.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

void lock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    printf("Process %d waiting for lock...\n", getpid());

    fcntl(fd, F_SETLKW, &lock);
}

```

```

        printf("Process %d acquired lock\n", getpid());
    }

void unlock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLK, &lock);
}

int main()
{
    int fd;
    pid_t pid;

    fd = open("sync.txt", O_RDWR | O_CREAT | O_APPEND, 0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    pid = fork();

    lock_file(fd);

    if(pid == 0)
    {
        write(fd, "Child writing to file\n", 22);
        sleep(3);
    }
    else
    {
        write(fd, "Parent writing to file\n", 23);
        sleep(3);
    }

    unlock_file(fd);

    printf("Process %d released lock\n", getpid());

    close(fd);
}

```

```

    return 0;
}

```

OUTPUT:

```

Data at offset 30: LINUX
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc parent.c -o parent
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./parent
Process 1881 waiting for lock...
Process 1882 waiting for lock...
Process 1881 acquired lock
Process 1881 released lock
Process 1882 acquired lock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$

```

18. Implement a program that tests for deadlock scenarios using file locks. How can you detect and resolve deadlocks?

PROGRAM:

```

/*
Program to implement deadlock
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : deadlock.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

void lock_region(int fd, int start)
{
    struct flock lock;

    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = start;
    lock.l_len = 10;

    printf("Process %d trying to lock region starting at %d\n",
    getpid(), start);

    if(fcntl(fd, F_SETLKW, &lock) == -1)
    {
        perror("fcntl");
        exit(1);
    }
}

```

```

    }

    printf("Process %d locked region %d\n", getpid(), start);
}

//Function to unlock a region
void unlock_region(int fd, int start)
{
    struct flock lock;

    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = start;
    lock.l_len = 10;

    fcntl(fd, F_SETLK, &lock);
}

//Main function
int main()
{
    int fd;
    pid_t pid;

    fd = open("deadlock.txt", O_RDWR | O_CREAT, 0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    pid = fork();

    if(pid == 0)
    {
        lock_region(fd, 0);
        sleep(2);

        lock_region(fd, 20);

        unlock_region(fd, 20);
        unlock_region(fd, 0);
    }
    else
    {
        lock_region(fd, 20);
        sleep(2);
    }
}

```

```

        lock_region(fd, 0);

        unlock_region(fd, 0);
        unlock_region(fd, 20);
    }

    close(fd);
    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc deadlock.c -o deadlock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./deadlock
Process 1907 trying to lock region starting at 20
Process 1908 trying to lock region starting at 0
Process 1907 locked region 20
Process 1908 locked region 0
Process 1907 trying to lock region starting at 0
Process 1908 trying to lock region starting at 20
fcntl: Resource deadlock avoided
Process 1907 locked region 0
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ |

```

19. Show how to use O_APPEND and file locks together to guarantee atomic writes to a log file from multiple processes.

PROGRAM:

```

/*
Program to append log entries to a file
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : logfile.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

//Function declaration
void lock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_WRLCK;

```

```

        lock.l_whence = SEEK_SET;
        lock.l_start = 0;
        lock.l_len = 0;

        fcntl(fd, F_SETLKW, &lock);
    }

//Function declaration
void unlock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLK, &lock);
}

//Main function
int main()
{
    int fd;
    pid_t pid;
    char buffer[200];
    time_t now;

    fd = open("logfile.txt", O_WRONLY | O_CREAT | O_APPEND,
0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    pid = fork();

    for(int i = 0; i < 5; i++)
    {
        lock_file(fd);

        now = time(NULL);

        if(pid == 0)
            sprintf(buffer, "Child PID %d: log entry at %ld\n",
getpid(), now);
        else

```

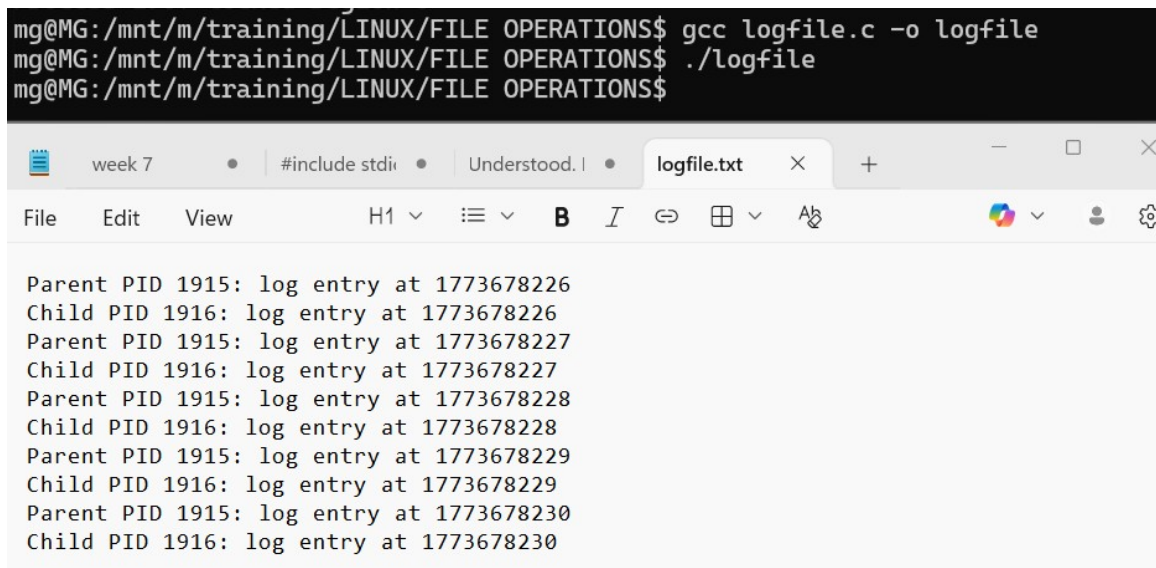


```

        sprintf(buffer, "Parent PID %d: log entry
at %ld\n", getpid(), now);
        write(fd, buffer, strlen(buffer));
        unlock_file(fd);
        sleep(1);
    }
    close(fd);
    return 0;
}

```

OUTPUT:



The screenshot shows a terminal window at the top with the following commands and output:

```

mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc logfile.c -o logfile
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./logfile
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$

```

Below the terminal is a text editor window titled 'logfile.txt'. It displays the output of the program, showing alternating log entries for Parent PID 1915 and Child PID 1916 over a period of 10 seconds (from 1773678226 to 1773678230).

```

Parent PID 1915: log entry at 1773678226
Child PID 1916: log entry at 1773678226
Parent PID 1915: log entry at 1773678227
Child PID 1916: log entry at 1773678227
Parent PID 1915: log entry at 1773678228
Child PID 1916: log entry at 1773678228
Parent PID 1915: log entry at 1773678229
Child PID 1916: log entry at 1773678229
Parent PID 1915: log entry at 1773678230
Child PID 1916: log entry at 1773678230

```

20. Write a program that uses `fcntl()` to query and display all current locks on a file.
How do you interpret the results?

PROGRAM:

```

/*
Program to check if a file is locked
Author   : MUTHUGANESH S
Date     : 03/02/2026
Filename : checklock.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
int main()
{
    int fd;
    struct flock lock;

```

```

fd = open("lockfile.txt", O_RDWR | O_CREAT, 0644);
if(fd < 0)
{
    perror("open");
    exit(1);
}
lock.l_type = F_WRLCK;
lock.l_whence = SEEK_SET;
lock.l_start = 0;
lock.l_len = 0;
if(fcntl(fd, F_GETLK, &lock) == -1)
{
    perror("fcntl");
    close(fd);
    exit(1);
}
if(lock.l_type == F_UNLCK)
{
    printf("No locks currently prevent this lock request.\n");
}
else
{
    printf("Lock detected:\n");
    printf("Process ID holding lock: %d\n", lock.l_pid);

    if(lock.l_type == F_RDLCK)
        printf("Lock type: READ LOCK\n");
    else if(lock.l_type == F_WRLCK)
        printf("Lock type: WRITE LOCK\n");

    printf("Start offset: %ld\n", lock.l_start);
    printf("Length: %ld\n", lock.l_len);
}
close(fd);
return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ gcc checklock.c -o checklock
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$ ./checklock
No locks currently prevent this lock request.
mg@MG:/mnt/m/training/LINUX/FILE OPERATIONS$

```